

UNIVERSITY OF CANBERRA · FACULTY OF SCIENCE & TECHNOLOGY

Version Control & Collaboration

11523 · Week 5 · Lecture 5

Git hands-on · branches · pull requests · merge conflicts · GitHub Actions & pre-commit

Dr Ibrahim Radwan Associate Professor of Machine Learning & AI

From Week 4 to Week 5

You can build, evaluate and track models. Now: how to manage the code itself — safely, and as a team.



Last week

Model evaluation & optimisation; tracking experiments with MLflow.



This week

Git and GitHub: history, branching, pull requests, conflicts and automation.



Why it matters

Every real data-science project is versioned and collaborative — and your Final Project will be too.

On the 14-week map: W4 Model Evaluation › **W5 Version Control & Collaboration** › W6 Containerisation & Deployment (Docker)

Tip: Git is a prerequisite for Week 6 — you'll version a Dockerised model, and CI will build it for you.

What we'll cover today



A

Why version control

The problem it solves;
centralised vs distributed.



B

Git essentials

init, add, commit, status, log,
diff, .gitignore.



C

Branching & merging

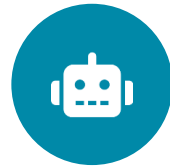
Branches, switching, merging
and resolving conflicts.



D

GitHub collaboration

Remotes, push/pull, pull
requests, reviews, teams.



E

Automation

GitHub Actions CI and pre-
commit hooks.



Follow along: open a terminal and run every command as it appears. One worked project — prediction.py — runs through the entire lecture, from first commit to automated CI.

By the end of this week you can...



Track your work

LO1

Initialise a repo and move changes through working → staging → repository.



Read project history

LO2

Use log, diff and show to inspect what changed, when and why.



Branch and merge

LO3

Work on parallel features and resolve merge conflicts confidently.



Collaborate on GitHub

LO4

Push, pull, open pull requests, review code and manage team access.

One project, followed all lecture

Rather than isolated commands, we build ONE real project from empty folder to automated pipeline — the rain-prediction model you met in Weeks 3–4.



The brief

You're the data scientist on a small team building a rain-prediction model (prediction.py). Over the next hour you'll take it from an untracked folder on your laptop to a reviewed, tested, automated repository on GitHub — hitting every real-world complication along the way.

Complications you'll hit (on purpose)

- A dataset you must NOT commit
- Urgent feedback mid-feature
- Two people edit the same function
- A push that gets rejected
- A teammate who needs to review you
Each one motivates the next Git concept — you'll never meet a command without first meeting the problem it solves.



Set up



Commit



Branch



Conflict



Share



Automate

Follow along in your own terminal — by the end you'll have this exact repository on your GitHub account.

PART A



Why Version Control?

The problem it solves — and the two kinds of system

Why version control?

Version control is a tool that manages different versions of your files and tracks every change made to them.



Undo & recover

Roll back an accidental change, or return to a version that worked last week.



Team collaboration

Several people edit the same code; Git combines everyone's work safely.



Parallel versions

Keep a stable production version while new features are built separately.

Without it: folders named final_v2_REALfinal.py, emailed zip files, overwritten work, and no way to answer “what changed, and why?” — enormous effort, and easy to get wrong.

Centralised vs distributed



Centralised (CVCS)

One copy of the repository on a central server; everyone commits to it.

- Single point of failure
- Server down = no history
- Example: Microsoft TFS



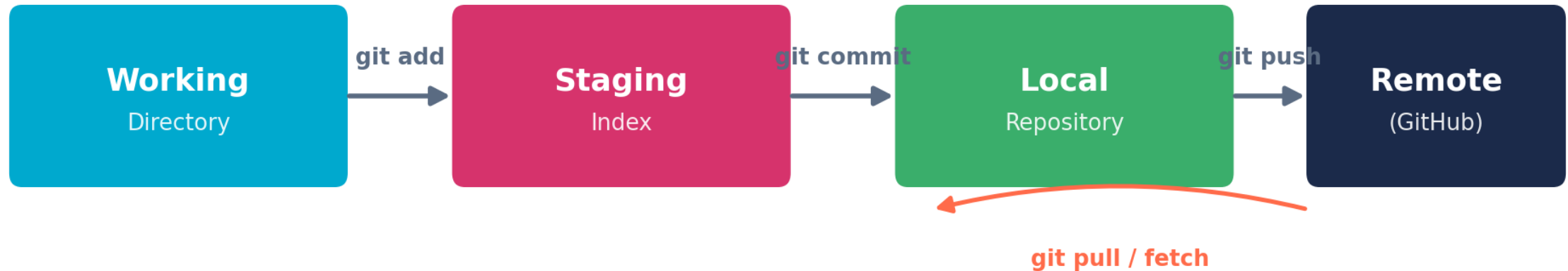
Distributed (DVCS)

Every client holds a FULL copy of the repository, history included.

- Work offline, commit locally
- No single point of failure
- Example: Git ← our focus

In practice: Git is distributed — you have the whole history locally, so you can commit, branch and read logs with no network at all.

The three areas — and how files move



Working Directory

Your files as you edit them. Git sees changes but isn't tracking them yet.



Staging Index

A holding area — you choose exactly what goes into the next commit.



Repository

Committed history. Permanent, logged, and shareable with your team.

PART B



Git Essentials

Setting up, tracking files, and reading your history

Step 1 — set up the rain-prediction repo

Tell Git who you are (it stamps every commit), then create a repository.

Terminal · configuration

```
# identify yourself (once per machine)
git config --global user.name "Your Name"
git config --global user.email "you@uni.edu.au"
git config --list          # check settings
```

Terminal · initialise

```
mkdir python_project
cd python_project
git init          # create the repo
git status        # what does Git see?
```

Expected output of git status

On branch master

No commits yet

nothing to commit (create/copy files
and use "git add" to track)

git init creates a hidden .git folder holding the entire repository — history, branches and config. Delete it and you delete the history (not your files).

Step 2 — commit prediction.py

- Create prediction.py, then run git status — Git reports it as untracked.
- git add moves it to the staging index (chosen for the next commit).
- git commit writes the staged snapshot into the repository, with a message.
- Run git status after each step to watch the file move between the three areas.

Terminal · add & commit

```
git add prediction.py    # one file
git add .                # everything

git commit -m "initial commit"
git status               # working tree clean
```

What you see along the way

```
# before add — untracked
Untracked files:
  prediction.py

# after add — staged
Changes to be committed:
  new file: prediction.py

# after commit
nothing to commit, working tree clean
```

Step 3 — the dataset you must NOT commit

- Some files should never be committed: data dumps, secrets, caches, model binaries.
- List them in a .gitignore file and Git stops tracking (and stops nagging about) them.
- Patterns work too: whole folders, or all files with an extension.
- Use `git restore --staged <file>` to move a file back out of staging.

File · .gitignore

```
# .gitignore
images/          # a whole folder
dataset/*.json   # by extension
__pycache__/*    *.pyc
.env             # secrets — never commit!
*.csv            data/
```

Data-science essentials to ignore

- Large datasets & raw data files
- Trained model binaries (.pkl, .h5)
- API keys, .env, credentials
- Notebook checkpoints & caches

Warning: committing a secret puts it in history permanently — even if you delete the file later. Add .gitignore BEFORE your first commit.

Step 4 — review what the team has done

Every commit gets a unique SHA hash, an author, a timestamp and your message. `git log` replays that history, newest first.

<code>git log</code>	full history, newest first
<code>git log --oneline</code>	one compact line per commit
<code>git log --reverse</code>	oldest first
<code>git log -p</code>	show the actual code changes
<code>git log --stat</code>	files changed + line counts
<code>git log -p <SHA></code>	that commit and earlier
<code>git show -p <SHA></code>	ONLY that commit
<code>git log --oneline --graph --all --decorate</code>	visual branch graph

`git log --oneline`

```
5429acd (HEAD -> master) study git log
4c75c75 initialise numpy and pandas
22ddd95 ignoring non-relevant files
7caa15a initial commit
```

HEAD is a pointer to where you are in history — it moves with every commit and every branch switch.

git diff — staged vs un-staged changes

- git diff shows changes in the working directory that are NOT yet staged.
- git diff --staged shows what IS staged, i.e. what your next commit will contain.
- Add a filename to limit it: git diff prediction.py

Reading a diff

- removed (old version)
+ added (new version)
@@ -1,2 +1,3 @@ old file: from line 1, 2 lines; new file: from line 1, 3 lines

git diff

```
diff --git a/prediction.py b/prediction.py
index 12ee3b4..d17a3fb 100644
--- a/prediction.py
+++ b/prediction.py
@@ -1,3 +1,5 @@
     import numpy as np
-    print('Check Git Logs')
+    print('Check')
```

PART C

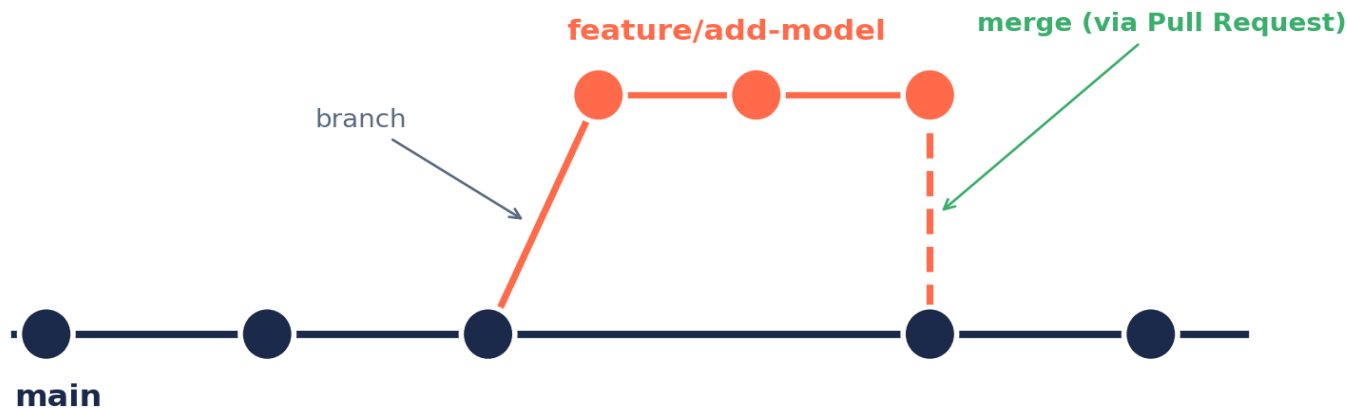


Branching & Merging

Working on parallel versions — and combining them

Step 5 — urgent feedback lands mid-feature

The situation: a client wants four features delivered in two phases — feature1 & feature2 first, then feature3 & feature4. While you're building phase 2, feedback arrives on phase 1 that must be fixed and shipped NOW — without releasing your half-finished phase-2 work.



- A branch is an independent line of development.
- **main** stays deliverable at all times.
- Feature work happens on its own branch.
- Merging brings the work back together.

Creating, switching and merging branches

<code>git branch</code>	list branches (* = current)
<code>git branch PhaseTwo</code>	create a branch
<code>git checkout PhaseTwo</code>	switch to it
<code>git switch PhaseTwo</code>	modern equivalent
<code>git checkout -b PhaseTwo</code>	create AND switch
<code>git merge PhaseTwo</code>	merge it into current branch
<code>git branch -d PhaseTwo</code>	delete a merged branch
<code>git branch v2 d16ba8c</code>	branch from an OLD commit

-am shortcut: `git commit -am "msg"` stages all TRACKED files and commits in one step (new files still need `git add`).

Terminal · the scenario

```
# work on phase two, isolated
git checkout -b PhaseTwo
git commit -am "Add Feature3 and Feature4"

# ship the phase-1 fix from main
git checkout master
git commit -am "fix issues in feature 1,2"
```

Then combine: switch to the branch you want to merge INTO, then merge the other one in.

```
git checkout master
git merge PhaseTwo
```

Step 6 — two people edited the same function

A conflict happens when the SAME lines changed on both branches — Git can't decide which version wins, so it asks you.

```
def predict():  
    <<<<<< HEAD (your current branch)  
        print('main version')  
    =====  
        print('feature version')  
    >>>>>> feature/add-model (incoming)
```

In VS Code: you get buttons — Accept Current / Accept Incoming / Accept Both / Compare. Same thing, friendlier.

- Git marks the clash inside the file with <<<<<< , ===== and >>>>>> .
- Above ===== is your current branch (HEAD); below it is the incoming branch.
- Edit the file: keep one side, the other, or a blend — then DELETE the markers.
- Stage the resolved file and commit to complete the merge.

Terminal · finish the merge

```
git add prediction.py  
git commit -am "merge phasetwo"
```

```
git merge --abort # bail out
```

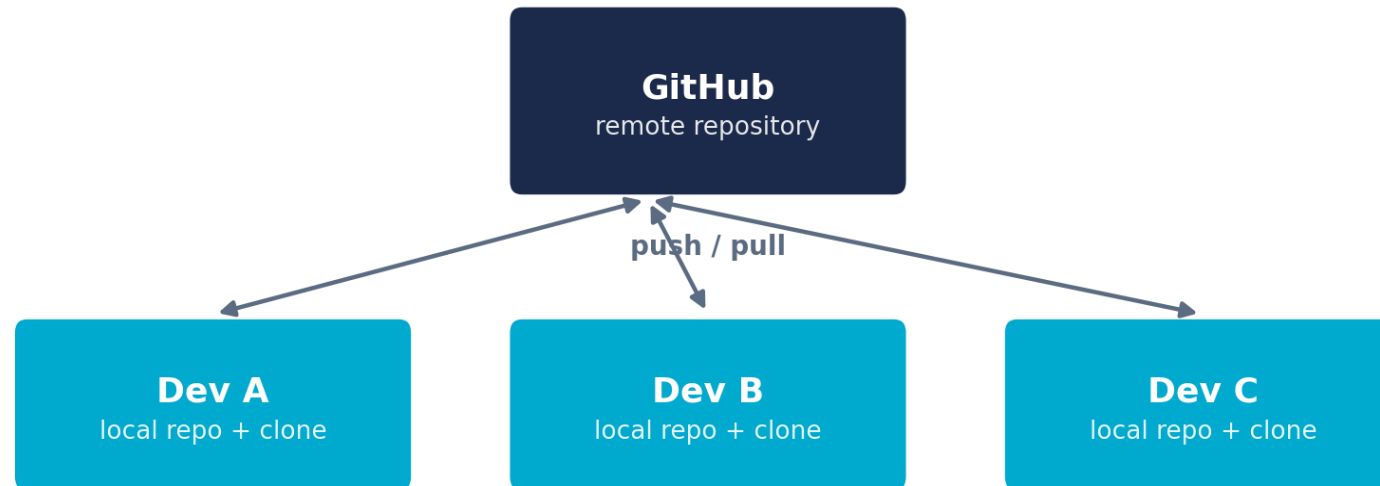
PART D



Collaborating on GitHub

From your laptop to a shared remote repository

Why put your repository on GitHub?



- Your laptop can fail — the remote is your backup.
- The team shares one source of truth.
- Code review happens through pull requests.
- It's your public portfolio for employers.



GitHub

The largest host; free public & private repos, Actions CI built in.



Collaboration

Issues, pull requests, reviews, discussions and project boards.



Portfolio

Recruiters look at GitHub profiles — your Final Project can live here.

Step 7 — publish the repo to GitHub

On github.com: click + → New repository, give it a name, choose Public or Private, and DON'T tick “Add a README” (your repo already has files).

1 Copy the repo's HTTPS URL from the green Code button

2 Link your local repo to that remote (name it origin)

3 Rename your branch to main (GitHub's default)

4 Push, with -u to remember the tracking link

Terminal · connect & push

```
# 2. connect local -> remote
git remote add origin \
    https://github.com/USER/prediction.git
git remote -v          # verify

# 3. GitHub's default branch name
git branch -M main

# 4. first push (-u sets upstream)
git push -u origin main
```

Authentication: GitHub no longer accepts passwords — use a Personal Access Token (Settings → Developer settings) or set up SSH keys.

Push, pull and staying in sync

<code>git clone <url></code>	copy an existing remote repo
<code>git push</code>	send your commits to the remote
<code>git push --all</code>	push every branch
<code>git pull</code>	fetch remote changes + merge
<code>git fetch</code>	fetch only — don't merge yet
<code>git remote -v</code>	show configured remotes
<code>git branch -a</code>	list local AND remote branches
<code>git checkout --track origin/dev</code>	work on a teammate's branch

A rejected push — and the fix

```
! [rejected] main -> main (fetch first)
```

hint: the remote contains work that you

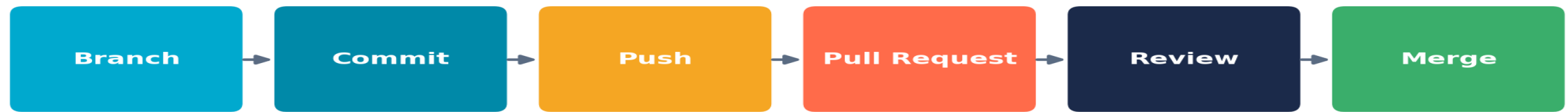
hint: do not have locally.

```
git pull      # get their changes
```

```
git push      # now it works
```

Golden habit: pull before you start work, and pull again before you push.
Most “Git disasters” are just someone who never pulled.

Step 8 — get your work reviewed (Pull Request)



A Pull Request (PR) proposes merging your branch into main, and opens it for review and discussion before anything is combined.



1 · Branch & push

git checkout -b feature/add-model,
commit, then git push -u origin
feature/add-model.



2 · Open the PR

GitHub shows a “Compare & pull
request” button. Add a title,
description and reviewers.



3 · Review

Teammates comment line-by-line,
request changes, or approve. CI checks
run automatically.



4 · Merge & tidy

Click Merge pull request, then delete
the branch. Pull locally to sync.

Working as a team & protecting main

Give teammates access

- Settings → Collaborators → Add people
- Invite by GitHub username or email
- Set the role: Read · Write · Admin
- For a private repo, only invitees can see it

Protect the main branch

- Settings → Branches → Add rule
- Require a pull request before merging
- Require approvals and passing CI checks
- Nobody (not even you) pushes to main directly

A simple, professional team workflow



Pull latest main



Branch per task



Commit small & often



Push & open PR



Review



Merge & delete

PART E



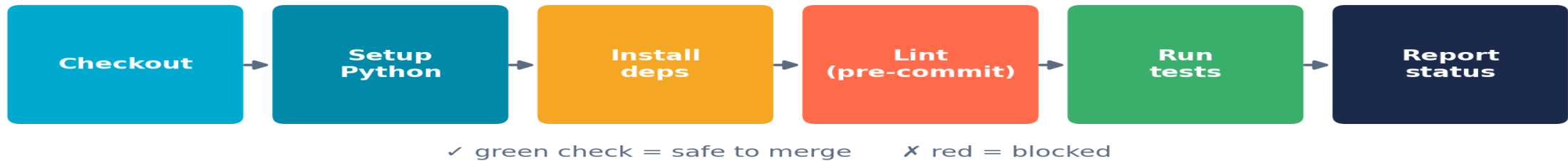
Automation

GitHub Actions and pre-commit — let the robots check your work

GitHub Actions — automate the checks

Actions runs a workflow automatically whenever you push or open a pull request: install, lint, test, report. If it fails, the PR is blocked.

push / pull_request → workflow triggers



Workflow

A YAML file in `.github/workflows/` describing what to run.



Trigger

on: push / pull_request — or a schedule.



Runner

A fresh cloud VM (ubuntu-latest) that executes your steps.



Status check

Green tick or red cross shown right on the pull request.

Step 9 — let CI test every push

Create `.github/workflows/ci.yml`, commit and push — Actions starts running immediately.

File · `.github/workflows/ci.yml`

```
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt
      - run: flake8 .
      - run: pytest -q
```

- name — what appears in the Actions tab.
- on — the events that trigger this workflow.
- runs-on — the machine image to use.
- uses — a pre-built action from the marketplace.
- run — any shell command you like.

For data science: add a step that runs your model's unit tests, or checks a notebook executes end-to-end.

Catch problems before they're even committed

- A hook runs automatically when you type git commit.
- The pre-commit tool manages these hooks from one config file.
- It can auto-format code, flag lint errors, strip notebook outputs and block secrets.
- If a hook fails, the commit is stopped — so bad code never enters your history.

Terminal · setup

```
pip install pre-commit
pre-commit install      # activate hooks

# run against everything once
pre-commit run --all-files
```

File · .pre-commit-config.yaml

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/psf/black
    rev: 24.4.2
    hooks:
      - id: black
  - repo: https://github.com/pycqa/flake8
    rev: 7.1.0
    hooks:
      - id: flake8
  - repo: https://github.com/kynan/nbstripout
    rev: 0.7.1
    hooks:
      - id: nbstripout
```

Git & GitHub cheat sheet

Cover the right column and try to recall what each command does.



Setup & basics

```
git init
git clone <url>
git status
git add .
git commit -m "msg"
git commit -am "msg"
```



History & compare

```
git log --oneline
git log -p
git show <SHA>
git diff
git diff --staged
git restore --staged <f>
```



Branch & remote

```
git branch -a
git checkout -b <name>
git merge <branch>
git push -u origin main
git pull
git remote -v
```

AVOID THESE

Common mistakes & how to fix them



Committing data or secrets

History is permanent. Add `.gitignore` FIRST; never commit `.env`, keys or big datasets.



Never pulling

Your push gets rejected and conflicts pile up. Pull before you start and before you push.



Giant vague commits

"update stuff" helps nobody. Commit small, logical changes with clear messages.



Working directly on main

Always branch. Protect main so a PR is required.



Panicking at a conflict

It's routine. Edit the markers, or run `git merge --abort` and retry.



Committing notebooks raw

Huge unreadable diffs. Use `nbstripout` via pre-commit.

Case study replay — all nine steps in order

Everything we did today, as one continuous sequence. This is the shape of real project work.

Local work · steps 1–5

```
# 1-2 set up and first commit
git init
git add prediction.py
git commit -m "initial commit"

# 3 keep data and secrets out
echo 'dataset/' >> .gitignore
git commit -am "add gitignore"

# 4 inspect the story so far
git log --oneline --graph --all

# 5 branch for the new feature
git checkout -b feature/add-model
git commit -am "add model"
```

Collaboration · steps 6–9

```
# 6 merge, resolve the conflict
git checkout main
git merge feature/add-model
# ...edit markers, then:
git add prediction.py
git commit -m "merge feature"

# 7 publish to GitHub
git remote add origin <url>
git branch -M main
git push -u origin main

# 8-9 PR + CI (on github.com)
git push -u origin feature/x
# open PR -> review -> Actions -> merge
```


Now run the whole case study yourself

Repeat the journey on your own project — the lab worksheet walks you through it, but try it unaided first.



Create & track

1

New folder, git init, add a .py file, commit it.



Ignore properly

2

Add a data/ folder and a .env; make sure NEITHER is committed.



Branch & merge

3

Branch, change a function, merge it back into main.



Force a conflict

4

Edit the SAME line on both branches, merge, and resolve it.



Publish & PR

5

Push to GitHub, open a pull request, merge it.



Automate

6

Add ci.yml so tests run on every push. Get a green tick.

Done all six? You now have the exact workflow expected in your Final Project — and the evidence of reproducibility the Paper Critique asks you to look for in others' work.

Key takeaways



Three areas, one flow

Working directory → staging (add) → repository (commit) → remote (push).



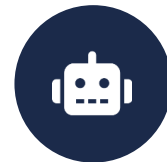
Branch for everything

Keep main shippable; do the work on a branch and merge it back deliberately.



PRs are collaboration

Push a branch, open a pull request, get it reviewed, then merge.



Automate the checks

GitHub Actions and pre-commit catch errors before they reach your team.

Your Week 5 — lab, study & assessment



In the lab

- Init a repo & commit prediction.py
- Branch, edit, merge — cause a conflict
- Push to GitHub & open a PR
- Add a CI workflow and watch it run



To study

- Re-run every command in these slides
- Explain staging in your own words
- Deliberately create & fix a conflict
- Set up pre-commit on your own project



Assessment

Paper Critique

In progress — due Week 9

Reproducibility is a criterion: check whether your paper's code is actually runnable

Final Project

Version your code on GitHub from day one

Further reading & references



Pro Git (Chacon & Straub) — free online

git-scm.com/book — the definitive reference; chapters 2 and 3 cover today's content.



GitHub Docs — Get started & Pull requests

docs.github.com — authentication, PRs, branch protection, collaborators.



GitHub Actions documentation

docs.github.com/actions — workflow syntax, triggers and the marketplace.



pre-commit.com

Hook catalogue and configuration guide, including nbstripout for notebooks.

The full Week 5 tutorial notes and the lab worksheet are on UCLearn (Canvas) under Week 5.

Questions?

Next week: Containerisation & Deployment — we'll Dockerise a model, versioned in the repo you just created.



Consultation

Wednesdays 13:30–14:30 · Rm 6C45
(or by appointment)



Contact

ibrahim.radwan@canberra.edu.au



Discussion forum

Ask on UCLearn — classmates see the answers too

See you in the lab.